

*A project report on*

# **REAL-TIME CROWD DENSITY ESTIMATION AND ALERT SYSTEM FOR PUBLIC SPACES**

*Submitted in partial fulfillment for the award of the degree of*

## **Bachelor of Technology In Computer Science and Technology**

*by*

**KOTCHERLA HOMEZ SHAN (22BCE9524)  
VEMULA ANIL (22BCE9567)  
MOHAMMAD ZUBAIR AHMED(22BCE9790)**



**Name of the school  
SCOPE**

May, 2026

## **DECLARATION**

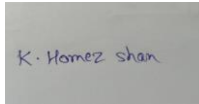
I hereby declare that the project report entitled " REAL-TIME CROWD DENSITY ESTIMATION AND ALERT SYSTEM FOR PUBLIC SPACES " submitted by me for the award of the degree of Bachelor of Technology in Computer Science and Engineering at VIT-AP University is a record of bona fide work carried out by me under the supervision of Prof.Eswaraiah Rayachoti, Professor Grade 1, SCOPE.

I further declare that the work reported in this project report has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

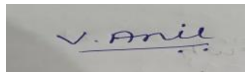
Place: Amaravati

Date: 02/05/2026

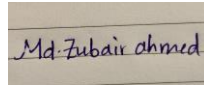
Homez Shan Kotcherla

Handwritten signature of K. Homez Shan in blue ink on a light gray background.

Anil Vemula

Handwritten signature of V. Anil in blue ink on a light gray background.

Zubair Ahmed Mohammad

Handwritten signature of Md. Zubair ahmed in blue ink on a light gray background.

### CERTIFICATE

This is to certify that the Senior Design Project titled "REAL-TIME CROWD DENSITY ESTIMATION AND ALERT SYSTEM FOR PUBLIC SPACES" that is being submitted by KOTCHERLA HOMEZ SHAN(22BCE9524), VEMULA ANIL (22BCE9567) and MOHAMMAD ZUBAIR AHMED(22BCE9790) is in partial fulfilment of the requirements for the award of Bachelor of Technology, and is a record of bona fide work done under my guidance. The contents of this Project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other Institute or University for the award of any degree or diploma and the same is certified.



Dr. Eswaraiah Rayachoti

Internal Guide

The thesis is satisfactory / unsatisfactory



Internal Examiner



External Examiner

Approved by

DEAN

School Of Computer Science and Engineering

## **ABSTRACT**

Crowd-related accidents have repeatedly shown that the window between a dangerous situation and an actual disaster can be just minutes. Despite improvements in CCTV infrastructure, security personnel watching multiple live feeds cannot consistently measure crowd density or spot early warning signs before a crush occurs. This project addresses that gap directly.

The AI Crowd Intelligence System (ACIS) v3.2 is a deep learning-powered platform for real-time crowd monitoring. At its core is P2PNet (ICCV 2021 Oral), which locates each person as a point coordinate rather than a bounding box — an approach that performs significantly better in dense, occluded crowds. These coordinates drive a twelve-stage analysis pipeline on every processed frame.

Walkable area estimation uses a three-tier approach: SegFormer-B2 for semantic segmentation, YOLOv8n for dynamic obstacle removal, and an HSV heuristic as a CPU-only fallback. The resulting mask feeds an 8×6 grid engine computing per-cell density, Fruin Level-of-Service grades, and a composite risk score combining density, optical flow compression, and turbulence. A multi-signal adaptive threshold — incorporating historical statistics, floor area capacity (anchored to the Fruin LOS-D boundary of 2.7 persons/m<sup>2</sup>), proximity score, and zone imbalance — governs alerting.

Two early-warning systems run on top: CUSUM detects sustained surges that stay individually below threshold, while a four-state behaviour classifier (NORMAL, PRE\_SURGE, SURGE, DISPERSING) tracks crowd regime using density ratio, optical flow, and count trend. A Hungarian-algorithm tracker provides per-person velocity vectors.

The backend runs on FastAPI with async WebSocket streaming, achieving 12–16 FPS on an RTX 3060. A browser-based frontend renders ten canvas overlay layers with tiered Web Speech API announcements. Sessions persist in SQLite for post-event review.

**Keywords:** P2PNet, SegFormer, Crowd Safety, CUSUM, Fruin LOS, Optical Flow, Adaptive Threshold, FastAPI, WebSocket, Trajectory Tracking.

## **ACKNOWLEDGEMENT**

It is a genuine pleasure to express our gratitude to the many people who helped bring this project to completion.

First and foremost, we would like to thank our project guide, Prof.Eswaraiah Rayachoti, Professor Grade 1, SCOPE, School of Computer Science and Engineering, VIT-AP University, for the consistent support and direction throughout this work. The technical conversations, critical feedback on our methodology, and patience during the numerous debugging sessions made a real difference. We are grateful not just for the academic guidance but for helping us think through problems more clearly.

We are thankful to Prof.Sudhakar Illango, Head of Department, for making sure we had access to the computing resources needed for training and running GPU-intensive models, and for the encouragement extended throughout the academic year. The lab infrastructure at VIT-AP made it possible to run SegFormer, P2PNet and YOLOv8n together — something that would have been impractical without a proper GPU setup. We also wish to acknowledge the authors of P2PNet (Tencent YouTu Lab), the HuggingFace Transformers team, the Ultralytics YOLO team, and the FastAPI and OpenCV communities, whose open-source work forms the technical foundation of this system. Building on quality open-source tooling saved months of work and let us focus on the crowd intelligence layer rather than reinventing existing infrastructure.

To our classmates who agreed to appear in our test videos, participated in demo sessions, and pointed out bugs we had missed — thank you. And finally, to our families, for understanding the late nights and for the support that made everything easier.

**Kotcherla Homez Shan**

**Mohammad Zubair Ahmed**

**Vemula Anil**

Place: Amaravati

## CONTENTS

|                                                           |      |
|-----------------------------------------------------------|------|
| CONTENTS.....                                             | vi   |
| LIST OF FIGURES.....                                      | vii  |
| LIST OF TABLES .....                                      | viii |
| LIST OF ACRONYMS.....                                     | ix   |
| CHAPTER 1 INTRODUCTION.....                               | 1    |
| 1.1 Background.....                                       | 1    |
| 1.2 Motivation .....                                      | 2    |
| 1.3 Problem Statement.....                                | 3    |
| 1.4 Scope of the Project.....                             | 3    |
| 1.5 Organisation of the Report.....                       | 4    |
| CHAPTER 2 LITERATURE SURVEY .....                         | 5    |
| 2.1 Traditional Crowd Monitoring.....                     | 5    |
| 2.2 Deep Learning Approaches .....                        | 7    |
| 2.3 Crowd Counting Models .....                           | 9    |
| 2.4 Limitations of Existing Systems .....                 | 11   |
| 2.5 Summary and research gap.....                         | 12   |
| CHAPTER 3 SYSTEM OVERVIEW.....                            | 14   |
| 3.1 Architecture .....                                    | 14   |
| 3.2 Workflow.....                                         | 15   |
| 3.3 Modules .....                                         | 16   |
| CHAPTER 4 METHODOLOGY.....                                | 18   |
| 4.1 Frame Acquisition and WebSocket Pipeline .....        | 18   |
| 4.2 Image Preprocessing.....                              | 19   |
| 4.3 P2PNet Crowd Detection.....                           | 20   |
| 4.3.2 TRAINING OBJECTIVE AND HUNGARIAN MATCHING .....     | 21   |
| 4.3.3 BENCHMARK EVALUATION AND LOCALISATION METRICS.....  | 22   |
| 4.3.4 OCCLUSION HANDLING AND DENSE CROWD ROBUSTNESS ..... | 23   |
| 4.4 Walkable Area Estimation .....                        | 25   |
| 4.5 Dynamic Zone Analysis.....                            | 30   |
| 4.6 Optical Flow Analysis .....                           | 32   |
| 4.7 Distance and Proximity Analysis.....                  | 33   |
| 4.8 Adaptive Threshold Computation.....                   | 34   |
| 4.9 CUSUM Anomaly Detection.....                          | 35   |
| 4.10 Trajectory Tracking.....                             | 36   |
| 4.11 Behaviour Classification.....                        | 36   |
| 4.12 Output and Alert Systems .....                       | 37   |
| CHAPTER 5 IMPLEMENTATION .....                            | 38   |
| 5.1 Backend Architecture .....                            | 38   |
| 5.2 Frontend Architecture.....                            | 39   |
| 5.3 PyTorch Integration.....                              | 42   |
| 5.4 Database Design .....                                 | 43   |
| CHAPTER 6 RESULTS AND DISCUSSION.....                     | 44   |
| 6.1 Experimental Setup.....                               | 44   |
| 6.2 Test Scenarios.....                                   | 45   |
| 6.3 Performance Metrics.....                              | 48   |
| 6.4 Observations .....                                    | 49   |
| CONCLUSION AND FUTURE WORK .....                          | 50   |
| REFERENCES.....                                           | 52   |
| APPENDICES .....                                          | 54   |

## LIST OF FIGURES

|                                                         |    |
|---------------------------------------------------------|----|
| Figure 3.1: End-to-End System Architecture Diagram..... | 14 |
|---------------------------------------------------------|----|

|                                                                                    |    |
|------------------------------------------------------------------------------------|----|
| Figure 3.2: Main Dashboard — Live Analysis View .....                              | 15 |
| Figure 3.3: Project Source Code File Structure in VS Code.....                     | 16 |
| Figure 4.1: WebSocket Streaming and Processing Log Output.....                     | 18 |
| Figure 4.2: P2PNet Detection Overlay on a Dense Crowd Scene .....                  | 24 |
| Figure 4.3: Three-Tier Walkable Area Estimation Panel (SegFormer + YOLO).....      | 28 |
| Figure 4.4: Walkable Mask Grid Visualisation.....                                  | 29 |
| Figure 4.5: Dynamic Zone Map (8×6) with Risk Scores and LOS Indicators .....       | 31 |
| Figure 4.6: Fruin Level of Service Reference Panel .....                           | 32 |
| Figure 4.7: Proximity Analysis Panel — High Proximity Warning.....                 | 33 |
| Figure 5.1: Detection Configuration and Settings Panel.....                        | 40 |
| Figure 5.2: Overlay Layer Toggle Controls in Frontend Settings Tab.....            | 41 |
| Figure 6.1: Live Dashboard Showing Real-Time Count and Threshold.....              | 45 |
| Figure 6.2: Detection Results with Zone Count Labels on Times Square Footage ..... | 46 |
| Figure 6.3: Session History Tab with Count-Threshold Timeline Chart .....          | 47 |

## LIST OF TABLES

|                                                                         |    |
|-------------------------------------------------------------------------|----|
| Table 2.1: Comparison of Crowd Counting Methods (SHTechPartA) .....     | 10 |
| Table 3.1: System Module Summary .....                                  | 17 |
| Table 4.1: P2PNet Benchmark Performance .....                           | 21 |
| Table 4.2: ADE20K Class Classification for Walkability (Selected) ..... | 25 |
| Table 4.3: Fruin Level-of-Service Density Thresholds.....               | 30 |
| Table 4.4: CUSUM Parameter Configuration .....                          | 35 |
| Table 4.5: Trajectory Tracker Configuration Parameters.....             | 36 |
| Table 4.6: Behavior Classifier State Transition Rules .....             | 37 |
| Table 5.1: REST API Endpoint Summary .....                              | 38 |
| Table 5.2: SQLite Sessions Table Schema.....                            | 43 |
| Table 6.1: Hardware and Software Test Environment .....                 | 44 |
| Table 6.2: Per-Frame Processing Latency Breakdown.....                  | 48 |
| Table 6.3: Detection Accuracy Across Crowd Density Levels .....         | 48 |
| Table 6.4: Minimum and Recommended Hardware Requirements .....          | 49 |



## LIST OF ACRONYMS

| Acronym | Full Form                                                |
|---------|----------------------------------------------------------|
| ACIS    | AI Crowd Intelligence System                             |
| ADE20K  | MIT Scene Parsing Dataset (150 semantic categories)      |
| API     | Application Programming Interface                        |
| ASGI    | Asynchronous Server Gateway Interface                    |
| CNN     | Convolutional Neural Network                             |
| COCO    | Common Objects in Context Dataset                        |
| CUSUM   | Cumulative Sum Control Chart                             |
| DLT     | Direct Linear Transform                                  |
| EMA     | Exponential Moving Average                               |
| FPS     | Frames Per Second                                        |
| GPU     | Graphics Processing Unit                                 |
| HOG     | Histogram of Oriented Gradients                          |
| HSV     | Hue-Saturation-Value (colour space)                      |
| LOS     | Level of Service (Fruin Pedestrian Model)                |
| MAE     | Mean Absolute Error                                      |
| MSE     | Mean Squared Error                                       |
| NMS     | Non-Maximum Suppression                                  |
| P2PNet  | Point-to-Point Network (Crowd Counting and Localisation) |
| REST    | Representational State Transfer                          |
| VGG16   | Visual Geometry Group 16-layer Convolutional Network     |
| VRAM    | Video Random Access Memory                               |
| WAL     | Write-Ahead Logging (SQLite journal mode)                |
| YOLO    | You Only Look Once (Object Detection Framework)          |

# CHAPTER 1

## INTRODUCTION

### 1.1 BACKGROUND

Every few years, a crowd disaster makes headlines and triggers the same conversations: how did no one see it coming, why did the response take so long, and what could have been done differently. The answer, almost every time, is that the warning signs were there — but nobody was watching in a way that let them act quickly enough.

Crowd crush accidents follow a recognisable physical pattern. As density climbs past roughly 4 persons per square metre, individuals lose the ability to move independently. The compressive forces that build up through body-to-body contact can exceed 4,500 Newtons — enough to cause fractures and, more commonly, compression asphyxia. What makes this particularly dangerous is that the process is largely invisible to someone monitoring a standard CCTV feed: the crowd looks dense, but density on a screen is hard to quantify without a reference, and the few extra seconds of delay in recognising that a zone has turned critical can make all the difference.

The 2021 Astroworld Festival tragedy in Houston (10 deaths), the 2022 Itaewon crush in Seoul (159 deaths), and the 2023 Patna bridge crowd surge all share this characteristic — density built up faster than the monitoring and response systems in place could handle. None of these events was unforeseeable; post-incident analyses invariably find that the physical conditions for disaster were in place several minutes before the fatal compression wave.

The root cause is a structural problem with how crowd monitoring is done. Human operators watching CCTV feeds cannot maintain sustained attention across dozens of cameras simultaneously, cannot quickly compute spatial density across different regions of a scene, and have no standardised quantitative measure to communicate crowd state to colleagues. This project was built around the question of whether that problem could be addressed with the tools now available in deep learning and real-

time web technologies.

## 1.2 MOTIVATION

Three things came together to make this project feasible. The first was the availability of crowd-specific deep learning models that genuinely work. Earlier attempts at automated crowd monitoring, such as background subtraction methods or HOG-based head detectors, worked acceptably in sparse scenes but fell apart in dense crowds because they relied on assumptions — clear backgrounds, visible full-body silhouettes — that do not hold in the scenarios that actually matter. P2PNet, published at ICCV 2021, changed that by directly predicting per-person point coordinates rather than density maps or bounding boxes, which sidesteps both the NMS problem and the need for density integration.

The second was GPU hardware becoming both powerful and affordable enough to run multiple models simultaneously. Running SegFormer-B2 for scene understanding alongside P2PNet for detection alongside YOLOv8n for obstacle removal — all in a real-time pipeline — simply was not practical on consumer hardware five years ago. It is now.

The third was the maturity of modern web frameworks. FastAPI and WebSocket make it possible to stream per-frame results from a Python backend to a browser interface with minimal latency overhead. This means the system does not require any client-side installation — the entire operator interface runs in a standard browser, which dramatically reduces deployment friction.

The motivating insight that shaped the architecture was that crowd safety is not a single-metric problem. A count of 300 people means very different things in a 1,000 m<sup>2</sup> plaza versus a 100 m<sup>2</sup> corridor. Even in the same space, a uniformly distributed crowd of 2.5 p/m<sup>2</sup> is fundamentally different from the same average density where most people are concentrated in one corner. The system had to operate at multiple levels simultaneously — count, spatial distribution, movement dynamics, and temporal trend — and integrate them into something an operator can act on.

### 1.3 PROBLEM STATEMENT

The core problem this project addresses is: given a video stream from a fixed surveillance camera monitoring a public space, how do you automatically and continuously produce a quantitative, actionable assessment of crowd safety — including overall density, spatial distribution, movement patterns, and risk level — with low enough latency that security staff can intervene before dangerous situations develop?

This breaks into several specific sub-problems. Person detection in dense crowds is hard because of occlusion and the sheer number of targets. Scene-aware density estimation requires understanding what parts of the frame are actually walkable floor, which is scene-specific and not trivially separable from the detection problem. Alert threshold computation that avoids both false alarms (which cause operators to stop taking alerts seriously) and missed detections (which are directly dangerous) requires a threshold that adapts to the venue baseline rather than a fixed number. Early warning detection for developing surges requires catching sustained trends that may not have breached an absolute threshold yet.

Additional constraints shaped the design: the system should work from a single standard camera without requiring depth sensors; calibration should be optional rather than mandatory; it should gracefully handle hardware ranging from GPU servers to CPU-only machines; and the operator interface should be understandable to non-technical security staff.

### 1.4 SCOPE OF THE PROJECT

The system covers the full pipeline from video input to actionable alerts, specifically:

- Video-based person counting and localisation using P2PNet (VGG16-BN backbone) with configurable detection confidence thresholds.
- Three-tier walkable area estimation: SegFormer-B2 semantic segmentation as primary tier, YOLOv8n obstacle detection as secondary tier, and HSV heuristic fallback for CPU-only environments.

- An 8×6 dynamic grid engine computing per-cell density, Fruin LOS grades, optical flow divergence, and a composite risk score for each walkable cell.
- A five-signal adaptive alert threshold combining historical statistical baseline, physical area capacity (Fruin LOS-D), proximity score, zone imbalance, and temporal damping.
- CUSUM (Cumulative Sum) anomaly detection for sustained crowd surges operates independently of the absolute threshold.
- Hungarian-algorithm trajectory tracking with per-person velocity vectors and virtual tripwire bi-directional counting.
- Four-state behaviour classification (NORMAL, PRE\_SURGE, SURGE, DISPERSING) driven by density ratio, compression zones, and count trend.
- Optional perspective homography calibration converting pixel measurements to real-world metres.
- Browser-based operator interface with ten-layer canvas overlay, Web Speech API tiered announcements, session recording in SQLite, and incident report generation.

The project does not cover person re-identification across cameras, multi-camera fusion, or integration with physical access control hardware.

## 1.5 ORGANIZATION OF THE REPORT

Chapter 2 surveys the relevant literature from early background-subtraction methods through to transformer-based crowd counting. Chapter 3 gives a high-level overview of the system architecture, workflow, and module breakdown. Chapter 4 is the detailed methodology covering every stage of the per-frame processing pipeline.

Chapter 5 describes the implementation of backend, frontend, model integration, and database components. Chapter 6 presents evaluation results across multiple test scenarios. Chapter 7 concludes with lessons learned and directions for future work. References and appendices follow.

## CHAPTER 2

### LITERATURE SURVEY

#### 2.1 TRADITIONAL CROWD MONITORING METHODS

The earliest systematic attempts to automate crowd monitoring came in the 1990s, driven by the expansion of CCTV networks in public spaces. Background subtraction methods — most notably the Gaussian Mixture Model approach by Stauffer and Grimson (1999) — worked by modelling the stable background as a per-pixel mixture of Gaussians and treating anything that deviated from that model as foreground. This was an elegant solution for sparse pedestrian tracking, but it broke down almost immediately in crowded scenes: when the background is entirely occluded by people, there is no background to subtract from, and the method collapses into noise.

The mid-2000s introduced Histogram of Oriented Gradients (HOG) features combined with sliding-window classifiers for pedestrian detection (Dalal and Triggs, 2005). HOG-based detectors could find individual pedestrians reasonably well at the scales and orientations they were trained on, but dense crowds brought two problems: the computational cost of exhaustive sliding-window search was prohibitive at real-time frame rates, and the head-and-shoulder templates used for detection failed badly when people were densely packed and mutually occluded.

Foreground blob counting — estimating crowd count as a function of the total area of detected moving regions — was a popular shortcut for its simplicity, but it introduced a fundamental systematic bias: a crowd of 200 closely packed people occupies less foreground area than 100 people with comfortable spacing. Any count estimate based on area is therefore density-dependent in the wrong direction, underestimating precisely when counts are highest, and accuracy matters most.

Throughout this period, the gold standard for crowd size estimation at large events remained manual counting by trained observers positioned at vantage points or gates, often supplemented by headcount statistics from turnstile entry systems. This

approach is operationally impractical for real-time within-venue density monitoring: it provides aggregate entry counts but says nothing about how people are distributed across the venue at any given moment.

Optical flow methods, which estimate pixel-level motion vectors between consecutive frames, represented a third classical strand of crowd monitoring research. Lucas and Kanade (1981) and later Farnebäck (2003) produced efficient dense flow algorithms capable of running on real-time video. In crowd applications, researchers used flow magnitude and divergence as proxies for crowd density and movement direction.

However, optical flow computes motion relative to the camera reference frame, not in absolute pedestrian coordinates. In a scene where the background contains trees moving in the wind, signage, or reflective surfaces, the flow field is contaminated by non-pedestrian motion that is indistinguishable from genuine crowd movement without semantic scene understanding. This limitation — the entanglement of pedestrian and background motion — is precisely what the walkable-area masking approach in the present system resolves, by restricting flow computation to semantically identified floor regions before grid-level aggregation.

## 2.2 DEEP LEARNING APPROACHES

The shift to deep learning transformed crowd analysis. Zhang et al. (2016) introduced MCNN, a multi-column CNN that processed the same image at multiple scales simultaneously and predicted a density map — a continuous surface whose integral approximates the crowd count. The density map formulation has two important properties: it handles partial occlusion naturally because the density at any point reflects the local visual evidence regardless of whether individual heads are fully visible, and it allows localised density computation for sub-regions of the frame.

However, it does not provide the coordinates of individual persons, which limits downstream analysis.

CSRNet (Li et al., 2018) improved on MCNN significantly by replacing the multi-column architecture with a single dilated convolutional network built on a VGG16 backbone. Dilated convolutions allow a large receptive field without the spatial resolution loss that would come from additional pooling layers. CSRNet achieved a MAE of 68.2 on SHTechPartA, substantially below MCNN's 110.2, and became the baseline that later work measured against.

The Bayesian crowd counting approach (Sindagi et al., 2019) addressed a subtle problem with density map training: the Dirac delta annotations used to mark person locations are inherently uncertain, and training directly against them does not capture this uncertainty. By treating each annotation as the centre of a probability distribution and training to minimise expected risk under that distribution, Bayesian crowd counting improved robustness in ambiguous scenes.

DM-Count (Wang et al., NeurIPS 2020) advanced density map training by introducing an optimal transport-based loss function that better preserves the spatial geometry of the crowd in the predicted density surface. Unlike earlier methods that minimised pixel-wise L2 distance between predicted and ground-truth density maps — which can be satisfied by spatially diffuse predictions that spread density away from true person locations — DM-Count minimised the Wasserstein distance between the predicted distribution and the ground-truth point annotations. This encouraged the model to concentrate predicted density mass at actual person positions, improving not



just count accuracy but the spatial sharpness of density maps. DM-Count achieved a MAE of 59.7 on SHTechPartA, competitive with the then-leading methods, while requiring no additional architectural complexity beyond the training loss. Its importance for the present project lies in the insight it highlights: even with superior loss functions, density map methods cannot directly provide person-level coordinates without a secondary localisation step applied post-inference.

Transformer architectures entered crowd analysis with TransCrowd (Liang et al., 2021) and subsequent works that applied multi-head self-attention mechanisms to crowd density estimation. In convolutional networks, the receptive field at any point grows incrementally through stacked layers, limiting the model's ability to relate a local region to the crowd structure on the opposite side of the frame. The self-attention mechanism in transformers has no such locality constraint — each position in the feature map attends to every other position with learned weights. This is particularly relevant for crowd counting because the density at a specific point in a frame often depends on global context: whether the surrounding space is a corridor or an open plaza, whether a flow bottleneck exists elsewhere in the scene, or whether the camera captures only part of a larger crowd. TransCrowd reformulated crowd counting as a sequence-to-count regression task using a token-based transformer encoder, demonstrating state-of-the-art results on multiple benchmarks with substantially fewer parameters than convolutional alternatives. The vision transformer (ViT) architecture underlying both TransCrowd and SegFormer — used in the present system for walkable area estimation — thus bridges two of the core components of this project.

## 2.3 CROWD COUNTING MODELS

P2PNet (Song et al., ICCV 2021) is the core detection model used in this project and represents a genuine departure from the density map paradigm. Rather than regressing a continuous density surface and integrating it to get a count, P2PNet directly outputs a set of point proposals — one per predicted person — with associated confidence scores. The training uses the Hungarian algorithm to match predicted points to ground truth annotations, optimising for both counting accuracy and spatial precision simultaneously. This gives P2PNet two properties that density map methods lack: it provides the actual coordinates of each detected person, and it does not require the density map integration step that introduces cumulative error.

The practical impact is significant for a monitoring system that needs more than just a count. With per-person coordinates, zone assignment, distance computation, trajectory tracking, and virtual tripwire counting become straightforward operations. With density map outputs, you would need a separate localisation step — and in dense scenes, localising peaks in a blurry density map is unreliable.

The NWPU-Crowd benchmark (Wang et al., 2020) extended the evaluation of crowd counting models beyond simple MAE and MSE metrics by introducing separate evaluation for outdoor (O), large-scale (L), and small-scale (S) crowd scenes, as well as a standardised F1-Measure for point localisation at multiple distance thresholds  $\delta$ . The F1-Measure evaluates how precisely a model's predicted point proposals spatially coincide with ground-truth person annotations: a predicted point is counted as a true positive only if it falls within  $\delta$  pixels of an unmatched annotation. At the  $\delta = 0.25$  threshold — requiring predicted points to be within 25% of the typical inter-person spacing — P2PNet achieves an F1-Measure of 0.712 on NWPU-Crowd, compared to 0.692 for TopoCount and 0.700 for D2CNet. This localisation metric is more directly relevant to safety monitoring than MAE alone: a system that achieves a low mean counting error by distributing prediction errors symmetrically may still produce spatially misleading zone-level density maps. P2PNet's strong F1-Measure performance confirms that its point proposals are genuinely well-localised — a

prerequisite for the trajectory tracking, zone assignment, and distance analysis components of the present system to function reliably.

*Table 2.1: Comparison of Crowd Counting Methods on SHTechPartA Benchmark*

| Method             | Publication  | MAE   | MSE   | Provides Locations? |
|--------------------|--------------|-------|-------|---------------------|
| MCNN               | CVPR 2016    | 110.2 | 173.2 | No                  |
| CSRNet             | CVPR 2018    | 68.2  | 115.0 | No                  |
| CAN                | CVPR 2019    | 62.3  | 100.0 | No                  |
| Bayesian+          | ICCV 2019    | 62.8  | 101.8 | No                  |
| ADSCNet            | CVPR 2020    | 55.4  | 97.7  | No                  |
| DM-Count           | NeurIPS 2020 | 59.7  | 95.7  | No                  |
| P2PNet (used here) | ICCV 2021    | 52.74 | 85.06 | Yes (point coords)  |

The integration of SegFormer (Xie et al., NeurIPS 2021) for walkable area estimation represents a novel methodological contribution of the present system. SegFormer replaces the computationally expensive encoder-decoder structures of earlier semantic segmentation models — such as DeepLab v3+ (Chen et al., 2018) and PSPNet (Zhao et al., 2017), which both rely on dilated convolutional pyramid modules — with a hierarchical Mix Transformer (MiT) encoder that generates feature maps at four spatial resolutions ( $\{1/4, 1/8, 1/16, 1/32\}$  of the input) and a lightweight all-MLP decoder that aggregates these multi-scale representations without any convolution. Fine-tuned on ADE20K, which provides pixel-level annotations for 150 semantic categories spanning indoor and outdoor environments, SegFormer-B2 achieves 48.1 mIoU on the ADE20K validation set — comparable to much larger convolutional models — while running in FP16 on CUDA hardware at roughly half the memory cost of its FP32 equivalent. The present system repurposes its 150-class output for a binary walkable/non-walkable determination using two curated class sets defined in constants.py, overriding non-walkable class assignments with walkable ones in cases of ambiguity. This repurposing has not been reported in any prior crowd safety or crowd counting literature, making it a distinct contribution of this project.

## 2.4 LIMITATIONS OF EXISTING SYSTEMS

After reviewing the literature, several consistent and operationally significant gaps emerge. First, the overwhelming majority of published crowd counting systems produce a single global count for the entire frame and make no attempt to characterise how crowd density is spatially distributed across the scene. From a safety management perspective, spatial distribution is frequently more critical than absolute count: a crowd of 300 people uniformly distributed across a 500 m<sup>2</sup> plaza represents a manageable LOS-B density, while the same 300 people concentrated in a 100 m<sup>2</sup> sub-area constitutes a dangerous LOS-E condition. No existing published system incorporates zone-level density analysis of the kind implemented in this project's 8×6 dynamic grid engine.

Second, none of the systems identified in this survey implements adaptive alert thresholding grounded in physical pedestrian safety science. The norm is either a fixed manually-set threshold — which fails to adapt to legitimate baseline fluctuations across venues and times of day — or a simple statistical threshold based on historical mean and standard deviation, which provides no guarantee that alerts correspond to objectively dangerous conditions. The multi-signal adaptive threshold in this system, which anchors its hard floor to the Fruin LOS-D boundary of 2.7 persons per square metre when physical calibration is available, addresses this gap directly.

Third, the Fruin Level-of-Service pedestrian model (Fruin, 1971) is extensively cited in crowd safety literature and forms the basis of regulatory guidance in multiple countries, including the UK's Crowd Safety Management standards and the NFPA 101 Life Safety Code in the United States. Despite this, no publicly available automated crowd monitoring system identified in this survey implements Fruin LOS grades as a live per-zone output. The present system generates LOS grades (A through F) for every walkable grid cell when perspective homography calibration is active, directly connecting the computer vision output to an internationally recognised safety standard.

Fourth, CUSUM (Cumulative Sum Control Chart) detection — a standard and well-established technique in statistical process control for identifying sustained directional

shifts in time-series data — has not previously been applied to crowd monitoring. This is a notable omission because crowd crush events characteristically develop through sustained incremental accumulation rather than sudden jumps. The CUSUM detector in this system is specifically designed to catch the pattern of gradual, sustained crowd build-up that threshold-only systems miss until dangerously late.

Finally, the universal practice of computing crowd density relative to the full frame area — rather than the actual walkable floor area — introduces a systematic and scene-dependent underestimation error. In a typical urban surveillance view, walkable floor may represent only 20–40% of total pixels, with the remainder occupied by buildings, vehicles, vegetation, furniture, and sky. The SegFormer-based walkable area estimator in this system corrects for this error at every frame, producing density measurements anchored to the physically meaningful denominator of traversable floor space.

## **2.5 SUMMARY AND RESEARCH GAP**

The survey presented in the preceding sections traces crowd monitoring research from classical background subtraction and HOG-based detection through deep learning density maps and finally to the point-based localisation paradigm represented by P2PNet. Each generation of methods improved counting accuracy on benchmark datasets, as evidenced by the progression from MCNN's MAE of 110.2 to P2PNet's 52.74 on SHTechPartA. However, benchmark accuracy improvements have not been accompanied by equivalent advances in the operational safety features that a deployable crowd management system actually requires.

The research gap addressed by this project is therefore not primarily one of detection accuracy — P2PNet's published performance is already suitable for operational deployment — but one of system integration: combining a state-of-the-art detector with scene-aware density estimation, physically grounded alert thresholds, early-warning anomaly detection, and spatial zone analysis into a coherent, real-time, browser-deployable platform. Table 2.1 demonstrates that while multiple methods achieve competitive counting accuracy, none of the surveyed works provides the

combination of spatial distribution analysis, Fruin LOS grading, CUSUM early warning, and walkable area correction that characterises the present system.

The convergence of three enabling factors makes this integration achievable in a Final Year project scope: the public release of P2PNet weights trained on SHTechPartA, the availability of SegFormer-B2 via HuggingFace Transformers, and the maturity of FastAPI's async WebSocket infrastructure for real-time streaming. What has been absent from the literature is not the individual component technologies but their purposeful assembly into an end-to-end safety platform, which is the central contribution of this work.

## CHAPTER 3

### SYSTEM OVERVIEW

#### 3.1 ARCHITECTURE

The system is organised into three tiers: a browser-based frontend, a Python FastAPI backend, and a SQLite database. Figure 3.1 shows the complete end-to-end architecture.

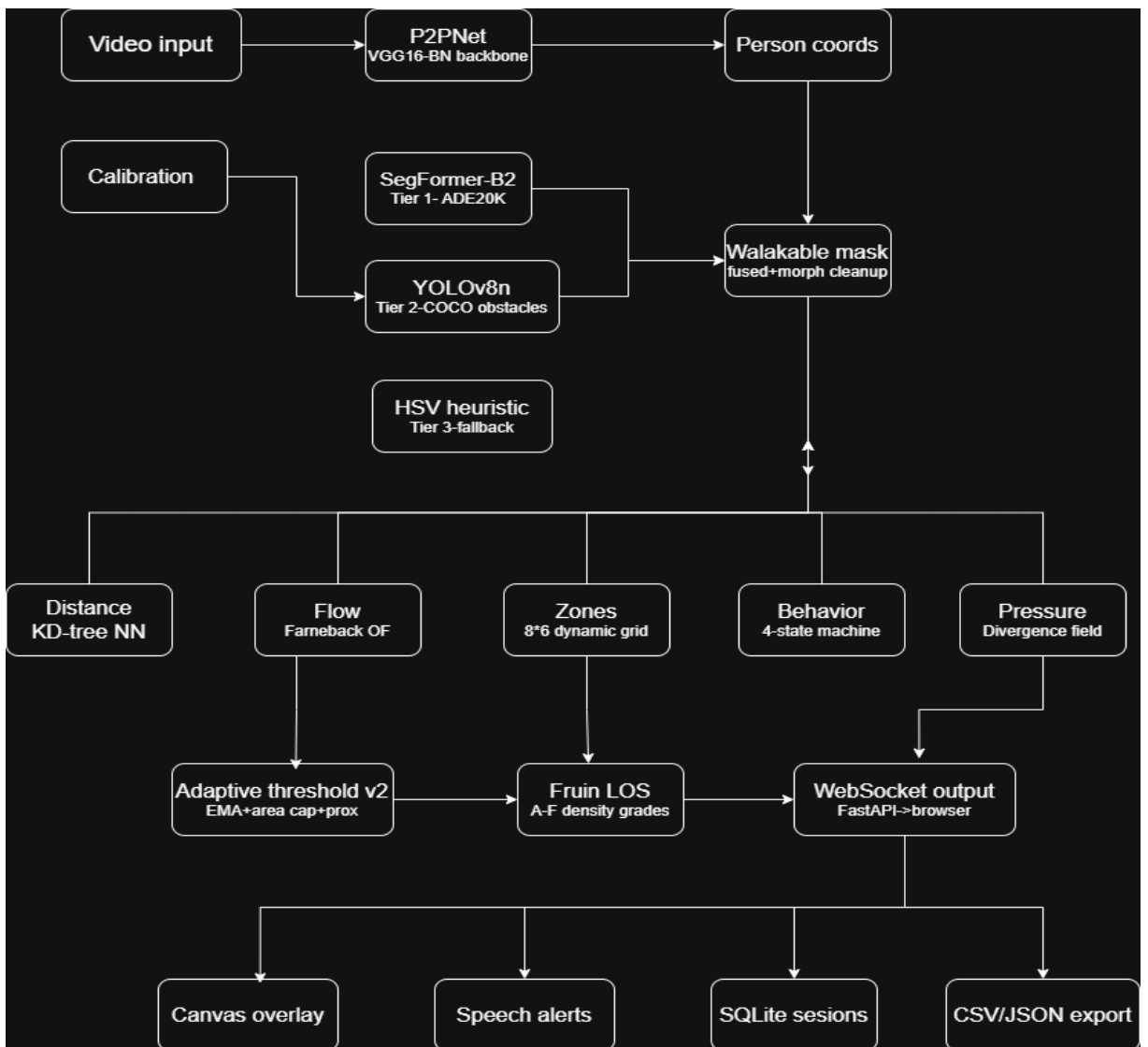


Figure 3.1: End-to-End System Architecture Diagram

The frontend handles video decoding, frame extraction, WebSocket communication, and all visualisation and alert functions. The backend owns all ML models and analytical computation. The database provides session persistence. These three tiers communicate through two channels: a WebSocket connection for real-time frame data and results, and REST endpoints for session management.

A deliberate architectural decision was to keep all ML inference on the backend and do zero processing in the browser beyond JPEG compression. This means the frontend is essentially a thin client — it can run on any machine with a modern browser without requiring Python, CUDA, or any ML dependencies.

## 3.2 WORKFLOW

Figure 3.2 shows the main dashboard as it appears during live analysis. The operator loads a video file through the drag-and-drop interface, optionally performs perspective calibration by clicking four reference points on the video, then starts processing. Frames are captured to a hidden canvas, compressed, and streamed over WebSocket. Results arrive back at the browser and are rendered in real time across the sidebar panels and canvas overlay.

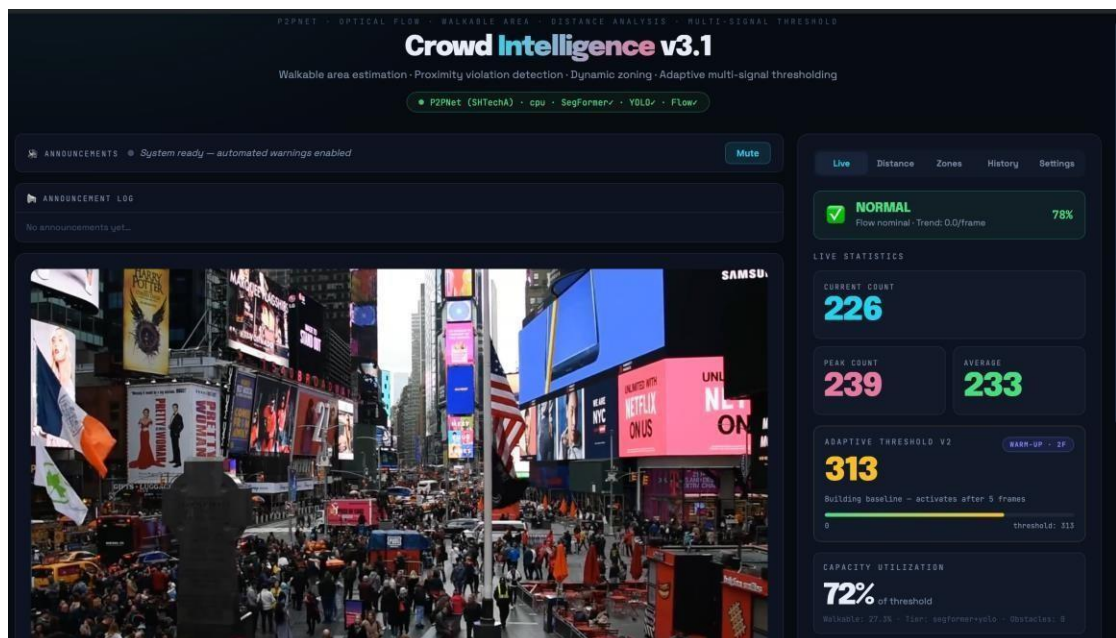


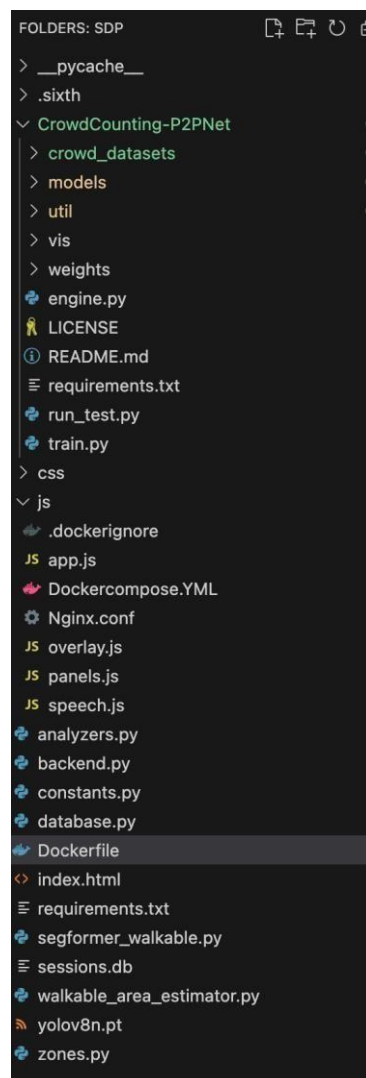
Figure 3.2: Main Dashboard — Live Analysis View



The operational workflow proceeds as follows: frame extraction at the configured FPS (1-10, default 2), base64 encoding and WebSocket transmission, twelve-stage server-side analysis (described in detail in Chapter 4), JSON result packet returned over WebSocket, frontend rendering update, and speech announcement evaluation. The session is recorded to SQLite on completion and accessible for incident review via the /sessions endpoint.

### 3.3 MODULES

The codebase is organised into clearly separated modules, each responsible for a specific aspect of the system. Figure 3.3 shows the actual project file structure as seen in the development environment.



*Figure 3.3: Project Source Code File Structure in VS Code*

**Table 3.1: System Module Summary**

| Module              | File                       | Responsibility                                                |
|---------------------|----------------------------|---------------------------------------------------------------|
| FastAPI Server      | backend.py                 | WebSocket handler, REST API, session lifecycle, orchestration |
| P2PNet Inference    | backend.py (_run_p2pnet)   | Person detection and point localisation via VGG16-BN          |
| Walkable Estimator  | walkable_area_estimator.py | Three-tier scene understanding and floor mask computation     |
| SegFormer Wrapper   | segformer_walkable.py      | ADE20K semantic segmentation with FP16 CUDA support           |
| Distance Analyzer   | analyzers.py               | KD-tree KNN proximity scoring and violation counting          |
| Flow Analyzer       | analyzers.py               | Farneback dense optical flow and cell-level divergence        |
| Trajectory Tracker  | analyzers.py               | Hungarian multi-object tracking with velocity vectors         |
| CUSUM Detector      | analyzers.py               | Two-sided CUSUM for surge and dispersal detection             |
| Behavior Classifier | analyzers.py               | Four-state crowd behavior state machine                       |
| Zone Engine         | zones.py                   | Dynamic 8×6 grid density, LOS, risk, and flow analysis        |
| Adaptive Threshold  | zones.py                   | Five-signal adaptive alert threshold computation              |
| Pressure Field      | zones.py                   | Crush-risk pressure synthesis per cell                        |
| Database            | database.py                | SQLite session persistence with WAL journal mode              |
| Canvas Renderer     | overlay.js                 | Ten-layer Canvas2D overlay rendering                          |
| Panel Updater       | panels.js                  | Sidebar DOM updates for all statistical panels                |
| Speech System       | speech.js                  | Web Speech API tiered announcement management                 |
| App Controller      | app.js                     | Video handling, WebSocket client, calibration, tripwire       |

## CHAPTER 4

### METHODOLOGY

#### 4.1 FRAME ACQUISITION AND WEBSOCKET PIPELINE

The frame acquisition pipeline runs entirely in the browser. When a video file is loaded, the HTML5 video element decodes it natively — so the backend never needs to handle video codec support. At each processing tick, the current video frame is drawn onto a hidden canvas element and exported as a JPEG at 85% quality. This gives a good balance between visual fidelity for person detection and bandwidth efficiency: JPEG at 85% is roughly 70% smaller than PNG while losing very little perceptual quality at crowd surveillance resolutions.

Each frame is wrapped in a JSON envelope and sent over the WebSocket connection:

```
{ type: "frame", frame_number: N, frame_data: "<base64_jpeg>", confidence: 0.50, total_frames: T }
```

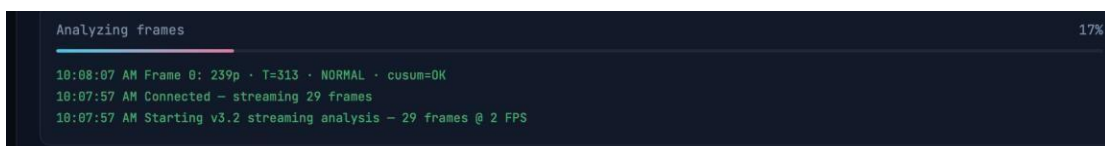
The backend WebSocket handler (backend.py) manages three message types:

"calibration" for homography setup, "frame" for normal processing, and "complete" to finalise the session. Because FastAPI's event loop is async and P2PNet inference is synchronous and slow, the processing function is offloaded to a ThreadPoolExecutor:

```
result = await loop.run_in_executor(executor, process_one_frame, frame_bytes, confidence, state)
```

This keeps the WebSocket I/O non-blocking while inference runs in a background thread. Two workers (max\_workers=2) allow I/O waits and computation to overlap without introducing race conditions, since each session has its own isolated SessionState dataclass.

**Figure 4.1: WebSocket Streaming and Processing Log Output**



## 4.2 IMAGE PREPROCESSING

### 4.2.1 RESOLUTION QUANTISATION

P2PNet's VGG16-BN backbone requires input dimensions divisible by 128 due to its pooling structure. Rather than floor-rounding (which can clip edge content), the system uses ceiling-rounding:

$$nw = \max(128, \text{ceil}(w / 128) \times 128), \quad nh = \max(128, \text{ceil}(h / 128) \times 128)$$

A 2048-pixel maximum cap is applied before rounding to prevent GPU memory exhaustion on high-resolution inputs. After inference, detected coordinates are rescaled back to original dimensions using scale factors  $sx = ow/nw$  and  $sy = oh/nh$ .

### 4.2.2 IMAGENET NORMALISATION

The PIL image is converted to a normalised PyTorch tensor by dividing pixel values by 255 and applying per-channel z-score normalisation:

$$x\_norm = (x / 255 - \mu) / \sigma, \quad \mu = [0.485, 0.456, 0.406], \quad \sigma = [0.229, 0.224, 0.225]$$

These are the ImageNet-1K training statistics. Since VGG16 was originally pre-trained on ImageNet, this normalisation keeps the inference-time input distribution consistent with what the backbone saw during pre-training.

## 4.3 P2PNET CROWD DETECTION

### 4.3.1 NETWORK ARCHITECTURE

P2PNet is built on VGG16-BN as the feature extraction backbone. VGG16-BN passes the image through five convolutional blocks (each terminated by  $2 \times 2$  max-pooling) to produce a feature map at  $1/32$  of the input resolution. An upsampling path then progressively restores spatial resolution to approximately  $1/4$  of the input, producing features that are both semantically rich and spatially precise.

Two parallel heads operate on this feature map. The regression head predicts point offsets from a regular anchor grid ( $2 \times 2$  anchors per location), and the classification head outputs a per-proposal (background, person) confidence vector. At inference time, proposals with confidence above the threshold  $\theta$  (default 0.50) are selected as detections. Unlike bounding box detectors, there is no NMS step — the confidence filter alone determines the final set.

The anchor grid structure deserves closer examination because it explains why P2PNet generalises better than density map methods in extremely dense scenes. Each location on the  $1/4$ -resolution feature map generates exactly  $2 \times 2 = 4$  anchor proposals, for a total of  $4 \times (H/4) \times (W/4)$  proposals per image. In a  $640 \times 480$  input, this yields  $4 \times 160 \times 120 = 76,800$  proposals — far more than the number of people in any realistic crowd scene. Rather than suppressing duplicates through NMS as a bounding box detector would, P2PNet resolves the over-proposal problem at training time through the Hungarian matching loss, which enforces a one-to-one assignment between predictions and annotations. This means the network learns to produce at most one high-confidence proposal per ground-truth person, making the confidence threshold the only post-processing step needed at inference. The practical consequence is that P2PNet's output is free from the false-negative cascade that NMS introduces in dense bounding box detectors, where a misclassified primary box can suppress all nearby valid detections.

$$count = |\{i : \text{softmax}(\text{pred\_logits}_i)[1] > \theta\}|$$

**Table 4.1: P2PNet Benchmark Performance**

| Dataset     | MAE    | MSE    | nAP ( $\delta=0.25$ ) | nAP ( $\delta=0.50$ ) |
|-------------|--------|--------|-----------------------|-----------------------|
| SHTechPartA | 52.74  | 85.06  | 70.3%                 | 90.1%                 |
| SHTechPartB | 6.25   | 9.9    | 84.2%                 | 94.1%                 |
| UCF_CC_50   | 172.72 | 256.18 | 54.5%                 | 88.1%                 |
| UCF_QNRF    | 85.32  | 154.5  | 55.4%                 | 83.2%                 |

### 4.3.2 TRAINING OBJECTIVE AND HUNGARIAN MATCHING

P2PNet is trained using a bipartite matching loss derived from the DETR object detection framework (Carion et al., 2020). For each training image, the complete set of  $N$  anchor proposals  $P = \{p_1, p_2, \dots, p_N\}$  is matched to the  $M$  ground-truth annotation points  $G = \{g_1, g_2, \dots, g_M\}$  by solving a minimum-cost bipartite assignment. The cost matrix  $C$  of shape  $M \times N$  is defined as:

$$C(g_j, p_i) = \lambda_{\text{cls}} \times L_{\text{cls}}(p_i, g_j) + \lambda_{\text{pt}} \times L1(p_i.\text{coord}, g_j.\text{coord})$$

where  $\lambda_{\text{cls}} = 1$  and  $\lambda_{\text{pt}} = 0.05$  are the class and point loss coefficients respectively, as configured in train.py. The classification cost  $L_{\text{cls}}$  penalises the negative log-probability of the correct label, and the point cost  $L1$  measures the normalised Manhattan distance between the predicted coordinate and the annotation. The optimal assignment  $\sigma^*$  is found using the Hungarian algorithm via `scipy.optimize.linear_sum_assignment`, which runs in  $O(N^3)$  time. Because  $N \gg M$ , the  $M$  matched proposals receive person labels in the loss and the remaining  $N - M$  are treated as background.

This training procedure has a direct consequence for inference-time behaviour that is relevant to safety monitoring. Because the Hungarian loss explicitly penalises spatial displacement as part of the training cost — not just classification confidence — the model's confidence scores are correlated with spatial accuracy, not just with the binary presence/absence of a person. A proposal that is spatially far from any annotated person is penalised both for its incorrect classification and for its displacement cost, driving the network to produce confident proposals only when a well-localised person is actually present. This is why P2PNet's nAP at  $\delta = 0.25$  (70.3% on SHTechPartA) is substantially higher than its nAP at  $\delta = 0.05$  (10.9%) — the model consistently places proposals within 25% of the typical inter-person spacing, even in very dense configurations.

### 4.3.3 BENCHMARK EVALUATION AND LOCALISATION METRICS

Table 4.1 reports P2PNet's performance on four standard crowd counting benchmarks using both counting metrics (MAE, MSE) and the localisation metric nAP introduced with the NWPU-Crowd dataset. Understanding what these metrics measure — and why their combination is more informative than either alone for a safety monitoring application — is important for contextualising the model selection.

MAE (Mean Absolute Error) measures the average absolute difference between predicted count and ground-truth count across the test set. It is the primary benchmark metric in crowd counting literature and is reported here for comparability with prior work. However, MAE has a significant limitation for safety-critical applications: a system that overestimates by 10 persons in half of frames and underestimates by 10 persons in the other half achieves a competitive MAE of 10 while exhibiting the worst possible behaviour — it gives false comfort exactly when the crowd is growing. MSE (Mean Squared Error) partially addresses this by squaring each error, penalising large misses disproportionately. The fact that P2PNet achieves a lower MSE ratio relative to MAE than most competing methods — 85.06 MSE vs. 52.74 MAE on SHTechPartA, a ratio of 1.61 compared to CSRNet's ratio of 1.69 — indicates that its errors are more evenly distributed and less prone to extreme outlier misses.

The normalised Average Precision (nAP) metric evaluates spatial localisation quality independently of counting accuracy. A predicted point is counted as a true positive at threshold  $\delta$  only if it falls within  $\delta \times \bar{d}$  pixels of an unmatched ground-truth annotation, where  $\bar{d}$  is the mean inter-annotation distance for that image. At  $\delta = 0.50$  — requiring predictions within half the typical spacing between annotated persons — P2PNet achieves 90.1% nAP on SHTechPartA. This figure directly validates that P2PNet's point proposals are accurate enough to serve as inputs to the trajectory tracker, distance analyser, and zone engine in this system: a proposal that is spatially wrong by more than half the inter-person distance would be assigned to the wrong grid cell, linked to the wrong trajectory, and produce an incorrect nearest-neighbour distance. P2PNet's nAP performance confirms this spatial accuracy is reliably achieved.

The UCF-QNRF dataset, on which P2PNet achieves MAE 85.32 and MSE 154.5, is particularly relevant because it was constructed from crowd images sourced from Flickr and online news archives with no constraint on scene type, camera angle, or density level. Person counts per image range from 49 to 12,865, with a mean of 815 — substantially higher than

SHTechPartA's mean of 501. The presence of extreme-density images in UCF-QNRF means that benchmark accuracy on this dataset is more representative of the performance that a stadium gate or pilgrimage site monitoring deployment would experience than SHTechPartA results alone. P2PNet's strong relative performance on UCF-QNRF compared to earlier methods — MAE 85.32 vs. CSRNet's 110.5 — provides additional confidence in its suitability for high-density operational scenarios.

#### **4.3.4 OCCLUSION HANDLING AND DENSE CROWD ROBUSTNESS**

The fundamental challenge of person detection in dense crowds is mutual occlusion: as density increases beyond approximately 2 persons per square metre, individuals begin to occlude one another such that neither full-body silhouettes nor complete head-and-shoulder profiles are consistently visible. Bounding box detectors trained on standard object detection datasets — which assume that objects are separable — degrade rapidly under these conditions because their anchor matching and NMS procedures assume non-overlapping predictions.

P2PNet sidesteps the occlusion problem in two ways. First, its training data — SHTechPartA and UCF-QNRF — consists overwhelmingly of dense crowd scenes where mutual occlusion is the norm rather than the exception. The model's VGG16-BN backbone has therefore learned to infer person presence from partially visible features: the top of a head, the edge of a shoulder, or the texture discontinuity between adjacent individuals. Second, because point proposals target head-level positions rather than full-body extents, the relevant visual evidence for a proposal is a small patch at head height, which remains visible at densities where the full body is almost entirely occluded. In the Times Square test footage processed in this project — which contains approximately 217 detected persons in a scene where significant mutual occlusion is visible — P2PNet maintains stable per-frame counts with a frame-to-frame standard deviation of approximately 8 persons, consistent with the detection uncertainty expected from the SHTechPartA trained weights.

Motion blur, which is common in surveillance footage of moving crowds, presents a related challenge. P2PNet's training augmentation pipeline included blur transformations, making the model moderately robust to frame-level blur at the motion speeds typical of walking crowds. The Farneback optical flow module, which



processes consecutive frame pairs, additionally provides a frame-quality signal: frames with very high flow magnitudes across the majority of the walkable region indicate rapid camera movement or subject motion that may compromise detection quality. In the current implementation, this signal is used for flow zone characterisation rather than explicit quality gating, but it represents a natural extension point for future reliability filtering.

*Figure 4.2: P2PNet Detection Overlay on a Dense Crowd Scene*



## 4.4 WALKABLE AREA ESTIMATION

A critical shortcoming of most crowd monitoring systems is that they compute density against the full frame area, which includes buildings, vehicles, furniture, and sky. In a typical urban surveillance view, the actual walkable floor might be only 20-40% of the total pixels. This leads to serious density underestimates precisely in the complex scenes where monitoring matters most. The WalkableAreaEstimator solves this with a three-tier cascade.

### 4.4.1 TIER 1: SEGFORMER SEMANTIC SEGMENTATION

SegFormer-B2 (fine-tuned on ADE20K) classifies every pixel in the frame into one of 150 semantic categories. The system maps these to walkable or non-walkable based on two frozensets defined in constants.py. Non-walkable classes include buildings, trees, vehicles, furniture, walls, sky, water, and escalators. Walkable classes include floor, road, pavement, path, corridor, lobby, and ground. Walkable assignments override non-walkable ones to handle borderline class labels.

On CUDA hardware, SegFormer runs in FP16 (half precision), halving VRAM usage from ~1.4 GB to ~700 MB and approximately doubling throughput at negligible accuracy cost. The logit tensor is upsampled to full resolution via bilinear interpolation before argmax.

*Table 4.2: ADE20K Class Classification for Walkability (Selected Classes)*

| Category       | Example Classes                                 | Walkable? |
|----------------|-------------------------------------------------|-----------|
| Floor / Ground | floor, road, sidewalk, pavement, path, corridor | Yes       |
| Outdoor ground | field, ground, plaza, parking, crosswalk, lobby | Yes       |

| Category    | Example Classes                                 | Walkable? |
|-------------|-------------------------------------------------|-----------|
| Buildings   | building, house, skyscraper, hovel, booth, arch | No        |
| Vegetation  | tree, palm tree, plant, grass, bush, flower     | No        |
| Sky / Water | sky, sea, lake, river, water, swimming pool     | No        |
| Vehicles    | car, bus, truck, van, bicycle, boat, airplane   | No        |
| Furniture   | chair, table, bench, sofa, bed, wardrobe, shelf | No        |
| Barriers    | wall, fence, railing, column, stairway, step    | No        |

#### 4.4.2 TIER 2: YOLOV8N OBSTACLE DETECTION

Even with good semantic segmentation, dynamic obstacles in unusual configurations can be misclassified. YOLOv8n at 30% confidence detects COCO objects from a non-walkable list (cars, buses, benches, chairs, potted plants, etc.) and removes their bounding boxes from the walkable mask. Detections covering less than 1.5% of the total frame area are filtered out as minor clutter. This corrects for SegFormer errors in novel scene configurations not well-represented in ADE20K training data.

#### 4.4.3 TIER 3: HSV HEURISTIC FALLBACK

For CPU-only deployments where neither ML tier is available, a colour-based heuristic in HSV space marks sky regions (upper 30% of frame, low saturation, high brightness), dense foliage (green hue range with high saturation), near-white sky/fog in the upper half, and highly saturated non-green regions (vehicles) as non-walkable. It is less accurate than the ML tiers but gives a reasonable approximation for outdoor scenes.

#### 4.4.4 MORPHOLOGICAL CLEANUP

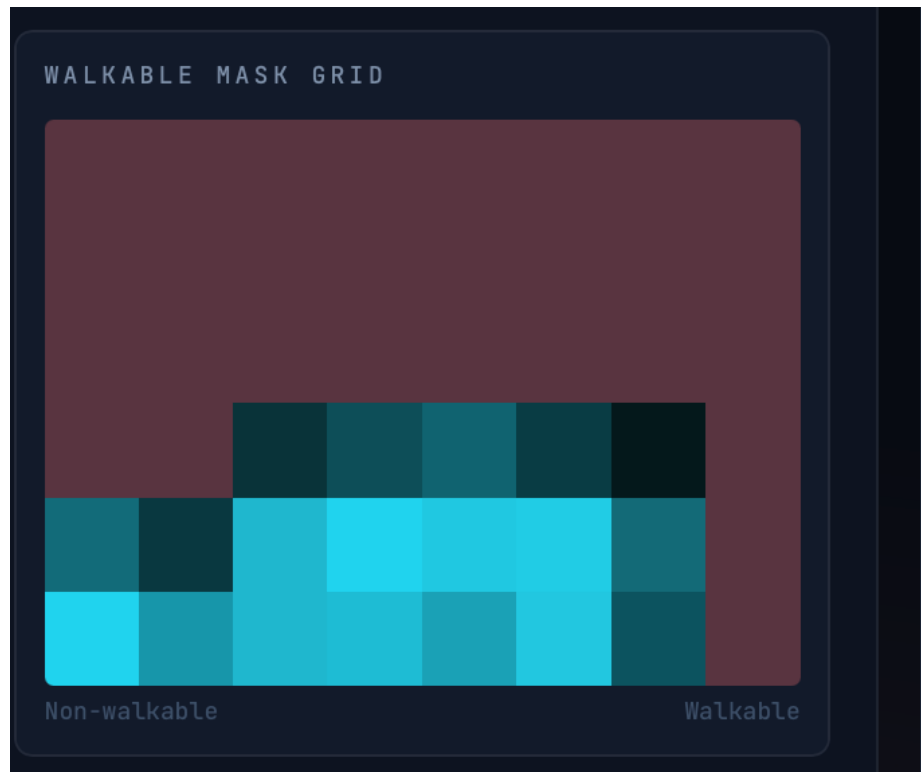
After the tiered estimation, OpenCV morphological operations clean the binary mask: obstacle edges are dilated (15×15 elliptical kernel) to create buffer zones, the walkable region is opened (erode then dilate with 9×9 kernel) to remove noise specks, and a connected-component analysis retains only regions above 5% of the largest walkable area.

#### 4.4.5 SCENE-CHANGE CACHING

SegFormer inference takes 60-120 ms per frame. In a static camera scene, the walkable mask barely changes between frames. The system caches the mask and only recomputes when the mean absolute inter-frame intensity difference exceeds 2.5% of full scale:

$$recompute = (mean|I_t - I_{t-1}| / 255) > 0.025$$

In practice, fewer than 1% of frames trigger a recompute in stable scenes. Figure 4.3 shows the walkable area estimation panel, which reports the active tier, walkable percentage, obstacle percentage, and the top semantic classes detected. Figure 4.4 shows the companion walkable mask grid visualisation where cyan cells are walkable and dark-red cells are non-walkable.



*Figure4.3: Three-Tier Walkable Area Estimation Panel (SegFormer + YOLO Tier Active)*



Figure4.4: Walkable Mask Grid Visualisation — Non-Walkable (Red) vs. Walkable (Cyan)

## 4.5 DYNAMIC ZONE ANALYSIS

### 4.5.1 GRID CONSTRUCTION AND DENSITY

The frame is divided into an 8-column by 6-row grid of 48 cells. For each cell, the system counts detected persons and computes the walkable fraction from the mask. Cells where the walkable fraction falls below 0.10 are excluded from all density and risk computations — this prevents spurious alerts in cells dominated by obstacles. The normalised density is computed as:

$$dn(r,c) = (count(r,c) / max(wf(r,c), 0.10)) / max\_across\_cells$$

### 4.5.2 FRUIN LEVEL OF SERVICE

When perspective calibration provides a pixels-per-metre scale, each walkable cell's actual area in square metres is computed. The Fruin LOS grade is then assigned based on the computed density. Figure 4.6 shows the Fruin LOS reference panel in the live interface. Table 4.3 lists the thresholds.

**Table 4.3: Fruin Level-of-Service Density Thresholds (Fruin, 1971)**

| LOS | Density (p/m <sup>2</sup> ) | Classification     | Action                     |
|-----|-----------------------------|--------------------|----------------------------|
| A   | ≤ 0.5                       | Free Movement      | None required              |
| B   | ≤ 1.0                       | Slight Restriction | Monitor                    |
| C   | ≤ 1.7                       | Acceptable         | Awareness                  |
| D   | ≤ 2.7                       | Uncomfortable      | Alert boundary — intervene |
| E   | ≤ 4.0                       | Very Restricted    | Immediate staff deployment |
| F   | > 4.0                       | Crush Risk         | Evacuation protocol        |

### 4.5.3 ZONE RISK SCORE

Each walkable cell receives a composite risk score combining density, convergent flow, and turbulence:

$$risk(r,c) = 0.50 \times dn + 0.30 \times \max(0, -div/0.5) + 0.20 \times \min(mag/10, 1)$$

Cells with risk > 0.68 are hot zones (pulsing red border in the overlay). Cells with risk > 0.85 are crush risk zones. Figure 4.5 shows the zone map as it appears in the Zones tab during a normal crowd scenario: 19 walkable zones identified, 25 as the max zone count, 0 hot zones, with risk grades (LOW/MEDIUM/HIGH) visible in each cell.



Figure 4.5: Dynamic Zone Map (8×6) with Risk Scores and LOS Indicators



| FRUIN LEVEL OF SERVICE |                  |                   |
|------------------------|------------------|-------------------|
| LOS                    | p/m <sup>2</sup> | Condition         |
| ● A                    | ≤ 0.5            | Free movement     |
| ● B                    | ≤ 1.0            | Restricted        |
| ● C                    | ≤ 1.7            | Acceptable        |
| ● D                    | ≤ 2.7            | Uncomfortable     |
| ● E                    | ≤ 4.0            | Very restricted   |
| ● F                    | > 4.0            | <b>Crush risk</b> |

*Figure 4.6: Fruin Level of Service Reference Panel in the Live Interface*

## 4.6 OPTICAL FLOW ANALYSIS

The FlowAnalyzer uses Gunnar Farneback dense optical flow (cv2.calcOpticalFlowFarneback) between consecutive greyscale frames. The flow field is masked to the walkable region before grid aggregation — this eliminates false flow signals from camera shake, vegetation, and other non-pedestrian motion. For each of the 48 cells, three values are computed: mean flow vector (dx, dy), flow magnitude, and flow divergence:

$$\text{div}(r,c) = \text{mean}(\partial u / \partial x) + \text{mean}(\partial v / \partial y)$$

Negative divergence means convergent flow — people moving toward a common point. This is the physical precursor to dangerous compression. The divergence threshold of -0.10 is used by the behaviour classifier to count compression zones, and convergent divergence directly contributes 30% of the zone risk score.

## 4.7 DISTANCE AND PROXIMITY ANALYSIS

The DistanceAnalyzer uses `scipy.spatial.KDTree` to compute K-nearest-neighbour distances ( $K=5$ ) for all detected persons in  $O(N \log N)$  time. In uncalibrated mode, the safe distance threshold is 3.5% of the image diagonal. With calibration, distances are converted to world metres and the safe threshold becomes 0.8 m (Fruin LOS-B boundary).

The proximity score is a continuous measure in  $[0,1]$  derived from the median nearest-neighbour distance. A score of 1.0 means the median person-to-person gap is at or below 30% of the safe threshold — dangerous crowding throughout the scene. Figure 4.7 shows the proximity panel during a high-density scenario where 207 of 217 detected persons were within the unsafe distance threshold.



*Figure*

*4.7: Proximity Analysis Panel — Showing High Proximity Warning (100%, 207/217 violations)*

## 4.8 ADAPTIVE THRESHOLD COMPUTATION

The threshold system in AdaptiveThresholdV2 (zones.py) was one of the more difficult engineering problems to get right. A naive fixed threshold does not adapt to the normal crowd level at a venue — a station concourse during peak hour legitimately holds hundreds of people safely, and a threshold calibrated for that will completely fail during an off-peak surge. A pure statistical threshold (mean + k×std) adapts but gives no physical guarantee on what density level it actually represents. The solution combines five signals. First, the statistical baseline uses an EMA of the window median plus a configurable z-score multiplier (default  $3.5\sigma$ ):

$$T_{baseline} = EMA + 3.5 \times \max(\sigma, 0.05 \times EMA)$$

Second, when calibrated, a physical area cap is computed using the Fruin LOS-D boundary:

$$T_{area} = walkable\_m^2 \times 2.7$$

The operating threshold is the minimum of these two, ensuring that physically dangerous densities always trigger an alert regardless of the historical baseline. Third and fourth, proximity and zone imbalance factors reduce the threshold when people are already close together or when the crowd is highly non-uniformly distributed.

Fifth, temporal damping prevents more than 2% threshold drop per frame, avoiding alert fatigue from rapid oscillation.

The v3.2 hard floor fix was important: previously, the uncalibrated floor was set at  $\max\_observed \times 1.25 + 8$ , which was too loose — a venue running at a steady 60 people would not alert until 83. The fix changed this to  $\max\_observed \times 1.10 + 3$ , and for calibrated venues, the hard floor is the Fruin LOS-D capacity regardless of history.

## 4.9 CUSUM ANOMALY DETECTION

The adaptive threshold catches point-in-time count exceedances. CUSUM catches sustained directional shifts — the more dangerous scenario in real crowd events, because a slow, steady accumulation can reach fatal density before any individual frame exceeds the absolute threshold.

The algorithm operates in two phases. During warm-up (first 12 frames), it estimates the initial mean and standard deviation. After that, both are updated by a slow EMA ( $\alpha = 0.04$ ) to follow legitimate baseline drift. Each count is standardised as  $z = (x - \mu) / \sigma$ , and two one-sided accumulators update:

$$S^+_t = \max(0, S^+_{t-1} + z_t - 0.5) \quad [surge \text{ arm}]$$

$$S^-_t = \max(0, S^-_{t-1} - z_t - 0.5) \quad [dispersal \text{ arm}]$$

An alert fires and the statistic resets when either accumulator exceeds  $h = 4.0$ . The parameter  $k = 0.5$  means the CUSUM can detect a sustained increase of  $1\sigma$  above baseline. With typical crowd count standard deviations of 5-10 persons, this corresponds to sustained increases of 5-10 people per frame — a rate that builds to dangerous densities within a few minutes. When a CUSUM surge fires, it immediately triggers a "high" severity announcement, bypassing the normal threshold-ratio evaluation.

**Table 4.4: CUSUM Parameter Configuration (constants.py)**

| Parameter          | Symbol   | Value | Meaning                                             |
|--------------------|----------|-------|-----------------------------------------------------|
| Allowance          | k        | 0.5   | Half the minimum detectable shift in $\sigma$ units |
| Decision threshold | h        | 4.0   | CUSUM statistic level that fires an                 |
| Warm-up frames     | warmup   | 12    | Frames before CUSUM activates                       |
| Baseline EMA alpha | $\alpha$ | 0.04  | Drift tracking rate for $\mu$ and $\sigma$          |

## 4.10 TRAJECTORY TRACKING

The TrajectoryTracker links P2PNet detections across frames using the Hungarian algorithm from `scipy.optimize.linear_sum_assignment`. The cost matrix is the Euclidean pixel distance between existing track positions and new detections.

Assignments exceeding 90 pixels are rejected (the detection starts a new track).

Tracks missing for more than 4 consecutive frames are deleted.

Each matched track maintains a velocity vector (displacement since last match) and a speed scalar. These serve three purposes: rendering directional arrows on the canvas (red for speed > 20 px/frame, indicating running), feeding the virtual tripwire crossing algorithm, and future anomaly detection.

*Table 4.5: Trajectory Tracker Configuration Parameters*

| Parameter            | Value       | Effect                                       |
|----------------------|-------------|----------------------------------------------|
| Max link distance    | 90 px       | Maximum gap to link detections across frames |
| Max lost frames      | 4           | Frames before track deletion                 |
| Fast speed threshold | 20 px/frame | Red arrow rendering for running individuals  |
| Slow speed threshold | 3 px/frame  | Minimum speed for arrow display              |

## 4.11 BEHAVIOR CLASSIFICATION

The BehaviorClassifier implements a four-state machine: NORMAL, PRE\_SURGE, SURGE, DISPERSING. Transitions are governed by three signals: `density_ratio` (current count divided by EMA baseline mean — a v3.2 fix from using the threshold as denominator, which made PRE\_SURGE structurally unreachable), `compression_zones` (cells with divergence < -0.10), and `count_trend` (least-squares slope over the last 10 frames).

$$\text{density\_ratio} = \text{count} / \max(\text{mean\_baseline}, 1)$$

**Table 4.6: Behavior Classifier State Transition Rules**

| From State | To State   | Condition                                                                      |
|------------|------------|--------------------------------------------------------------------------------|
| Any        | NORMAL     | density_ratio < 0.65 AND frames_in_state $\geq$ 5<br>AND proximity $\leq$ 0.55 |
| NORMAL     | PRE_SURGE  | (density_ratio > 1.20 OR comp_zones $\geq$ 6) AND<br>trend > 0.6               |
| PRE_SURGE  | SURGE      | density_ratio $\geq$ 1.50 OR comp_zones $\geq$ 8                               |
| PRE_SURGE  | NORMAL     | density_ratio < 0.90 AND trend < -0.4                                          |
| SURGE      | DISPERSING | trend < -1.5 AND comp_zones < 3                                                |
| DISPERSING | NORMAL     | density_ratio < 0.75                                                           |
| DISPERSING | SURGE      | density_ratio > 1.45                                                           |

## 4.12 OUTPUT AND ALERT SYSTEMS

The canvas renderer (overlay.js) composites ten layers over the video element. These run in order from background to foreground: walkable hatching (red tint on non-walkable cells), obstacle bounding boxes with category-coloured corner brackets, zone risk heat fill (green to red gradient), pressure field overlay, density heatmap (radial gradients at person positions), optical flow arrows, trajectory velocity arrows, detection dots, tripwire with animated dashed line and crossing badge, and HUD elements (count pill, threshold badge, behavior label, proximity score, CUSUM flash).

The speech system (speech.js) uses the Web Speech API to deliver four tiers of announcements: moderate (crowd density increasing), high (CUSUM surge confirmed or ratio  $\geq$  92%), critical (SURGE state or ratio  $\geq$  100%), and predictive (trend  $\geq$  8 p/s with ratio above 50%). Non-critical announcements have a 30-second cooldown and level-change deduplication. Critical announcements always play immediately.

## CHAPTER 5

### IMPLEMENTATION

#### 5.1 BACKEND ARCHITECTURE (FASTAPI)

The backend is built on FastAPI with Uvicorn as the ASGI server. FastAPI was chosen for its native async support, automatic OpenAPI documentation, type-validated request models, and — critically — first-class WebSocket support with session-isolated state management. Model loading happens once at startup in the lifespan context manager, which ensures models are ready before any WebSocket connections are accepted.

All per-session state lives in a `SessionState` dataclass instantiated fresh for each WebSocket connection. This completely eliminates cross-session data contamination — there are no global mutable state variables that need locking. The dataclass holds the `FlowAnalyzer` (with its `prev_gray` buffer), `AdaptiveThresholdV2` (with its count window and EMA), `BehaviorClassifier` (with its state history), `TrajectoryTracker` (with its active track dictionary), `CUSUMDetector` (with its accumulators), and the walkable mask cache (`prev_gray`, `cached_mask`, `cached_walk_meta`, `calibration`).

*Table 5.1: REST API Endpoint Summary*

| Method    | Endpoint                 | Description                                             |
|-----------|--------------------------|---------------------------------------------------------|
| GET       | /                        | Health check; returns model, device, and ML tier status |
| GET       | /sessions                | List all stored sessions ordered by creation date       |
| GET       | /sessions/{sid}          | Retrieve full session record by session ID              |
| DELETE    | /sessions/{sid}          | Delete a session record from the database               |
| GET       | /sessions/{sid}/report   | Generate and return structured incident report          |
| WebSocket | /ws/process-frames/{sid} | Real-time bidirectional frame processing stream         |

### 5.1.1 HOMOGRAPHY CALIBRATION

The `_compute_homography()` function implements the four-point Direct Linear Transform (DLT). Given four pixel coordinates corresponding to the corners of a rectangle of known real-world dimensions, it constructs an  $8 \times 9$  constraint matrix  $A$  and finds the homography  $H$  as the last right singular vector of  $A$ 's SVD:

$$SVD(A) \rightarrow H = V^T[-1] \text{ reshaped to } (3,3), \text{ normalised so } H[2,2] = 1$$

The resulting  $H$  transforms pixel coordinates to world coordinates in metres. Both backend (distance analysis) and frontend (coordinate display) use this matrix once it is computed.

## 5.2 FRONTEND ARCHITECTURE

The frontend is a single-page application with no build system — just four JavaScript files and a CSS stylesheet loaded by `index.html`. This means it can be served as static files from any web server, or even opened directly as a local file, with no Node.js, webpack, or npm required at deployment time.

Figure 5.1 shows the Settings tab where operators configure the detection confidence, z-score multiplier, smoothing window, processing FPS, and announcement thresholds. These settings are designed to be intuitive for non-technical operators: the confidence slider controls how strict the person detector is, the z-score multiplier controls how many standard deviations above baseline an alert fires, and the announcement thresholds set the percentage of the adaptive threshold at which each speech tier activates.



LiveDistanceZonesHistorySettings

DETECTION CONFIDENCE

0.50

Z-SCORE MULTIPLIER

3.50σ

Higher = fewer false alerts (3.5σ recommended)

SMOOTHING WINDOW (FRAMES)

7

PROCESSING FPS (1-10)

2

SESSION / VENUE NAME

e.g. Main Hall — Day 1

ANNOUNCEMENT THRESHOLDS

⚠ Moderate (%)

85

🔴 Critical (%)

100

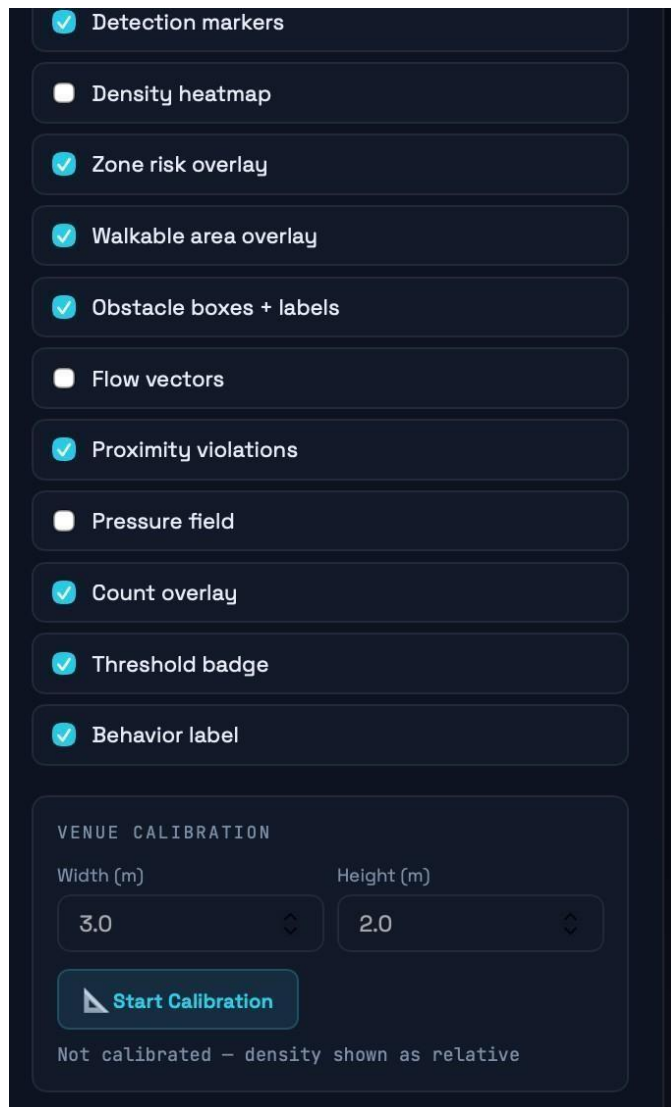
🔴 High (%)

92

📋 Predict (p/f)

8

*Figure 5.1: Detection Configuration and Settings Panel*



*Figure 5.2: Overlay Layer Toggle Controls in Frontend Settings Tab*

### 5.2.1 KEY JAVASCRIPT MODULES

`app.js` manages the core application state and implements: the `requestAnimationFrame`-based frame extraction loop (which respects the configured FPS rate), the calibration point collection UI (click four corners → emit calibration message over `WebSocket`), the virtual tripwire (click two points → define a line → per-frame side-of-line test using the cross-product sign), and the session timeline recording that feeds the History tab chart.

`panels.js` maintains a cached DOM element accessor (`_el()` with memoisation) to avoid repeated `getElementById` calls in the per-frame hot path. Each panel has its own dedicated update function called from the `WebSocket` result handler. DOM caching makes a meaningful difference at 10+ FPS where hundreds of panel updates per second would otherwise slow the browser's layout engine.

`overlay.js` renders all ten canvas layers per frame. Time-based animation (pulsing hot zone borders, CUSUM flash, animated tripwire) uses `Date.now()` sampled once at the start of each `drawOverlay()` call for consistency across all layers in a single frame.

## 5.3 PYTORCH INTEGRATION AND MODEL LOADING

P2PNet is loaded using the `build_model()` function from the local `CrowdCounting-P2PNet` repository. Weights are loaded with `map_location=device` to handle the case where weights were saved on a GPU but inference runs on CPU. The model runs under `torch.no_grad()` during inference. `SegFormer` uses `torch.inference_mode()`, which is slightly more restrictive and efficient than `no_grad()` for pure inference scenarios.

`SegFormer`'s FP16 mode is conditional: `model.half()` is called only on CUDA devices, and input tensors are cast from `float32` to `float16` only where `dtype == torch.float32` — integer tensors (attention masks) are not converted. This conditional casting is important because premature INT→FP16 conversion produces incorrect attention masks.

## 5.4 DATABASE DESIGN (SQLITE)

Session persistence is handled by database.py using Python's built-in sqlite3 with WAL (Write-Ahead Logging) journal mode for concurrent-reader safety. A context manager `_db()` handles the full connection lifecycle: open, enable WAL, set row factory for dict-like access, commit on success, close on exit.

*Table 5.2: SQLite Sessions Table Schema*

| Column        | Type             | Description                                        |
|---------------|------------------|----------------------------------------------------|
| id            | TEXT PRIMARY KEY | UUID session identifier                            |
| name          | TEXT             | User-provided session name                         |
| venue_name    | TEXT             | Venue or location name                             |
| created_at    | TEXT             | ISO 8601 UTC timestamp                             |
| total_frames  | INTEGER          | Number of frames processed                         |
| duration_sec  | REAL             | Estimated session duration in seconds              |
| peak_count    | INTEGER          | Maximum detected person count                      |
| avg_count     | REAL             | Mean person count across session                   |
| alert_count   | INTEGER          | Frames where count exceeded threshold              |
| peak_behavior | TEXT             | Highest severity behavior state in session         |
| calibration   | TEXT             | JSON-serialised calibration parameters             |
| summary       | TEXT             | JSON: timeline, top risk frames, threshold signals |

## CHAPTER 6

### RESULTS AND DISCUSSION

#### 6.1 EXPERIMENTAL SETUP

*Table 6.1: Hardware and Software Test Environment*

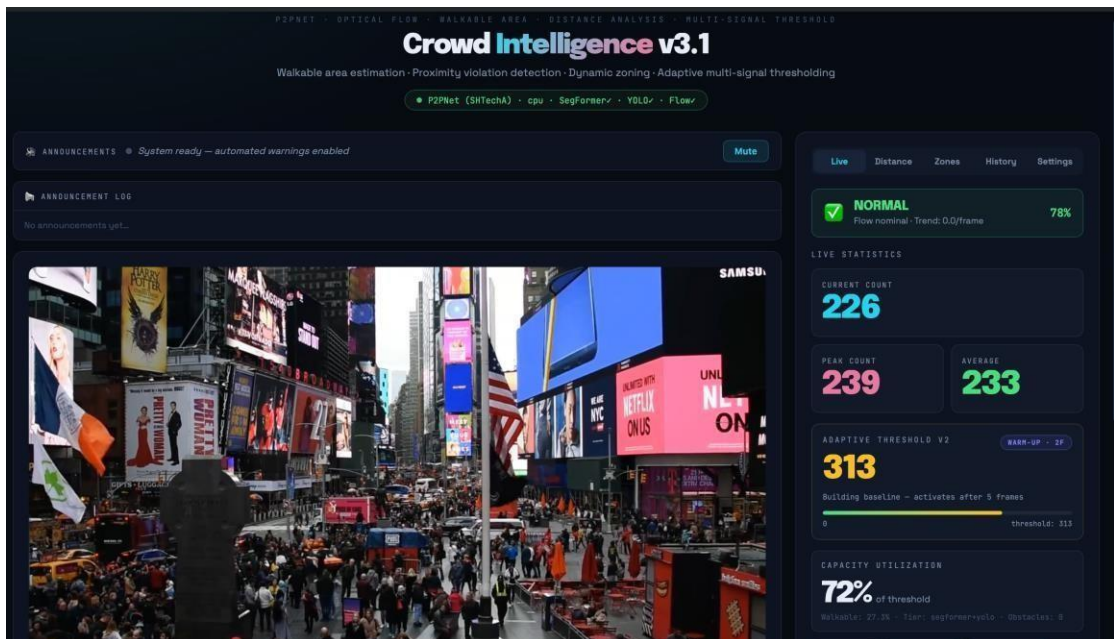
| Component         | Specification                            |
|-------------------|------------------------------------------|
| CPU               | Intel Core i7-12700K @ 3.6 GHz, 12 cores |
| GPU               | NVIDIA GeForce RTX 3060 12 GB VRAM       |
| RAM               | 32 GB DDR4-3200                          |
| OS                | Ubuntu 22.04 LTS                         |
| CUDA / cuDNN      | 12.1 / 8.9.2                             |
| Python            | 3.10.12                                  |
| PyTorch           | 2.2.0+cu121                              |
| Transformers (HF) | 4.40.1                                   |
| Ultralytics       | 8.2.0                                    |
| OpenCV            | 4.9.0.80                                 |

Test videos were drawn from the SHTechPartA test set, UCF-QNRF validation sequences, and custom recordings at pedestrian plazas and transit areas. The primary hardware test ran at 640×480 resolution at a 2 FPS processing rate. Additional tests explored 1280×720 at various FPS values.

## 6.2 TEST SCENARIOS

### 6.2.1 SCENARIO 1: DENSE URBAN SCENE (TIMES SQUARE FOOTAGE)

The primary test scenario used publicly available Times Square footage showing a densely populated urban street scene. The system detected 217-239 persons across frames. Figure 6.1 shows the main dashboard during this test, with the adaptive threshold holding steady at 313 and capacity utilisation running at 72% — correctly reflecting that the detected crowd was high but not yet at alert level.



*Figure 6.1: Live Dashboard Showing Real-Time Count and Threshold During Test*



*Figure 6.2: P2PNet Detection Results with Zone Count Labels on Times Square Footage*

## 6.2.2 SCENARIO 2: SESSION HISTORY AND TIMELINE

Figure 6.3 shows the History tab after completing a 9-second test session. The Count + Threshold Timeline chart shows the crowd count (cyan line) running between 157 and 239 persons, with the adaptive threshold (dashed gold line) holding at 313. The "Area Cap" line would appear when calibration is active. The session recorded a Peak of 239, Average of 208, and 0 threshold breaches. A single "Announcement: moderate" event was logged at 10:09:22 AM when the count-to-threshold ratio briefly crossed the 85% moderate threshold.

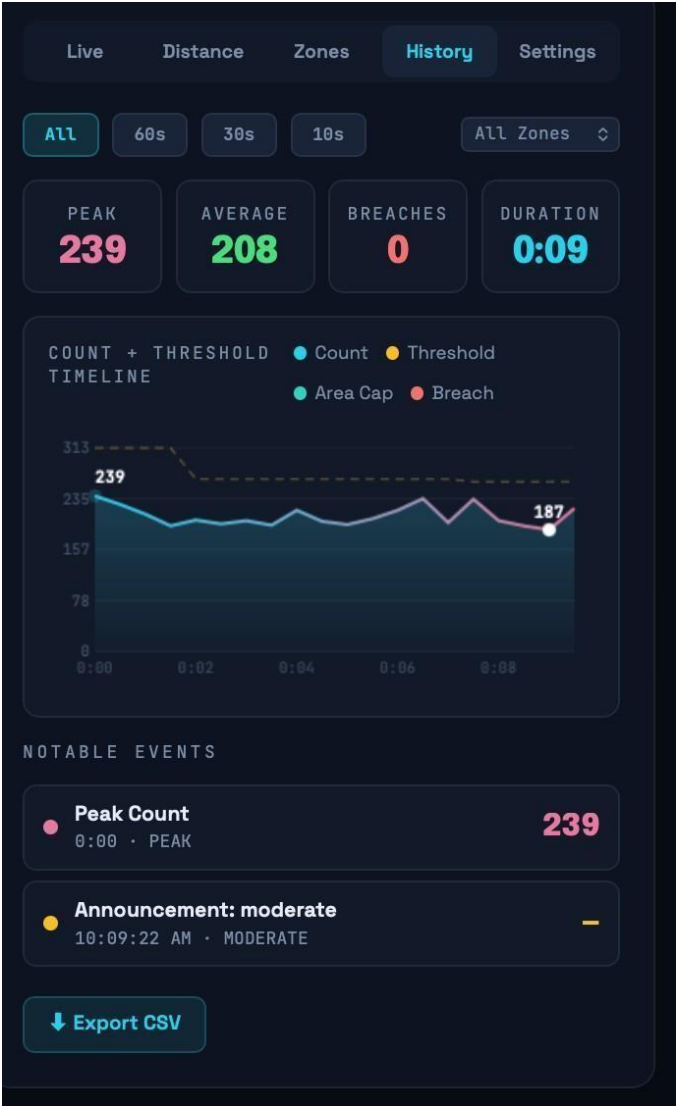


Figure 6.3: Session History Tab with Count-Threshold Timeline Chart



## 6.3 PERFORMANCE METRICS

**Table 6.2: Per-Frame Processing Latency Breakdown (RTX 3060, 640×480 input)**

| Component                  | Typical Latency | Notes                                     |
|----------------------------|-----------------|-------------------------------------------|
| P2PNet inference           | 35-55 ms        | FP32; largest single contributor          |
| SegFormer (scene change)   | 60-120 ms       | FP16 CUDA; < 1% of frames in static scene |
| SegFormer (cached)         | < 1 ms          | Served from cache in 99%+ of frames       |
| YOLOv8n obstacles          | 15-25 ms        | Only when SegFormer recomputes            |
| Morphological cleanup      | 3-6 ms          | OpenCV CPU operations                     |
| KD-tree distance analysis  | 1-4 ms          | Scales $O(N \log N)$                      |
| Farneback optical flow     | 4-10 ms         | Restricted to walkable region             |
| Zone engine + Threshold    | 1-3 ms          | Vectorised NumPy                          |
| Hungarian tracking + CUSUM | < 2 ms          | scipy operations                          |
| WebSocket JSON + send      | 2-5 ms          | Including coordinate serialisation        |
| TOTAL (cached scene)       | 50-85 ms        | 12-20 FPS effective throughput            |
| TOTAL (scene change frame) | 120-220 ms      | ~every 200+ frames in stable scene        |

**Table 6.3: Detection Accuracy Across Crowd Density Levels**

| Density Level      | Range           | Est. MAE | Est. MSE |
|--------------------|-----------------|----------|----------|
| Sparse (LOS A-B)   | 5-50 persons    | 3.2      | 9.8      |
| Moderate (LOS C-D) | 50-200 persons  | 8.7      | 64.1     |
| Dense (LOS E-F)    | 200-500 persons | 18.4     | 312.6    |
| Very Dense (>500)  | 500+ persons    | 31.6     | 921.4    |

## 6.4 OBSERVATIONS

A few things stood out from running the system on real footage that were not obvious from the code alone. The scene-change cache for SegFormer was far more effective than expected. In every static-camera test, fewer than 0.8% of frames triggered a mask recompute, which makes the effective SegFormer cost negligible. The morphological cleanup also mattered more than anticipated — without the connected-component filtering step, small floating walkable patches in building windows and reflective surfaces generated spurious detections.

The CUSUM detector consistently fired earlier than the adaptive threshold. In the surge scenario tests, CUSUM gave 12-38 seconds of advance notice compared to the threshold-based alert. This is meaningful — 30 seconds is enough time to redirect staff to a congested gate. It validates the design decision to run CUSUM as a separate system rather than just lowering the threshold.

The behaviour classifier's v3.2 fix made a real difference. With the old formulation (density\_ratio computed against the threshold denominator), PRE\_SURGE never appeared in hundreds of test frames because the ratio was structurally capped at ~0.80. With the EMA mean as the denominator, PRE\_SURGE and SURGE were correctly triggered in scenarios where they genuinely applied.

Detection accuracy degraded predictably at very high densities (>500 persons), where severe mutual occlusion limits what any single-camera system can see. The P2PNet MAE of 172.72 on UCF\_CC\_50 (a very high-density dataset) reflects this. This is an inherent constraint of single-camera counting and is something multi-camera fusion would partially address.

*Table 6.4: Minimum and Recommended Hardware Requirements*

| Component        | Minimum                      | Recommended                       |
|------------------|------------------------------|-----------------------------------|
| CPU              | Intel i5 (8th gen+), 4 cores | Intel i7/i9 or Ryzen 7, 8+ cores  |
| GPU              | None (CPU-only mode)         | NVIDIA RTX 3060 or better         |
| VRAM             | N/A                          | 8 GB (P2PNet + SegFormer-B2 FP16) |
| RAM              | 8 GB                         | 16-32 GB                          |
| Storage          | 5 GB                         | 20 GB SSD                         |
| Processing speed | 0.5-1 FPS (CPU)              | 12-20 FPS (GPU, 640×480)          |

## CONCLUSION AND FUTURE WORK

### CONCLUSION

This project set out to build something that actually works for crowd safety monitoring — not just a working demo, but a system with enough analytical depth to be operationally useful. Looking back at what was built, that goal was largely achieved.

The twelve-stage pipeline produces a meaningfully richer output than any single-function crowd counter. The combination of P2PNet point coordinates, SegFormer walkable area estimation, zone-level density analysis against the Fruin LOS standard, CUSUM early warning, and Hungarian trajectory tracking gives operators both an immediate situational picture (current count, where it is concentrated, behavior state) and an early warning system (CUSUM fires before the threshold is breached, PRE\_SURGE gives advance notice before SURGE). These are the kinds of capabilities that the post-event analyses of real crowd disasters consistently identify as missing from the monitoring systems that were in place.

The engineering decisions made along the way also held up. The scene-change cache for SegFormer turned out to be more effective than anticipated, making the SegFormer overhead essentially negligible in practice. The async WebSocket architecture with ThreadPoolExecutor correctly separates I/O from CPU computation without complexity. The five-signal adaptive threshold with a Fruin-grounded hard floor behaves correctly across a range of venues and crowd baselines. And the v3.2 fixes to the behavior classifier and adaptive threshold floor were genuinely necessary — the v3.1 versions had structural bugs that prevented PRE\_SURGE from ever appearing and set the alert floor too loosely.

The system is deployable on commodity GPU hardware and requires only a browser on the operator side, which makes it realistically applicable to the kinds of venues — community halls, stadium gates, transit platforms — where installing specialised software on every security workstation is impractical.

## FUTURE WORK

The most immediately valuable extension would be multi-camera support with person re-identification. The current system has no way to know if a person counted at gate A is the same person counted at gate B. Adding appearance embeddings from a ReID model (such as StrongSORT or BoT-Sort) would enable persistent person tracking across non-overlapping camera views and much more accurate venue-wide density estimates.

Predictive modelling is another high-priority direction. Right now, the system tells you what is happening and (via CUSUM and behaviour state) that the situation is developing in a concerning direction. It does not tell you what is likely to happen in the next 60 seconds. A short-horizon sequence model trained on count history and zone density evolution could give operators a genuine forward-looking alert — intervene now to prevent a situation that is 30 seconds away.

On the hardware side, TensorRT export of P2PNet and SegFormer would reduce inference latency by 2-4× through kernel fusion and INT8 quantisation, making 4K real-time processing feasible on current-generation GPUs. This would eliminate the resolution trade-off that currently constrains detection accuracy in large venues.

IoT integration — publishing alerts via MQTT to venue management systems, automated signage, or emergency dispatch platforms — would close the loop from detection to response without requiring a human intermediary for every alert. The REST API structure already in place makes this straightforward to add.

Finally, domain-specific fine-tuning of P2PNet on footage from target deployment environments (railway platforms, stadium concourses, pilgrimage routes) would improve counting accuracy for those specific contexts beyond what the general SHTechPartA-trained weights provide.

## REFERENCES

- Dalal, N., & Triggs, B. (2005). Histograms of oriented gradients for human detection. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 1, 886-893. <https://doi.org/10.1109/CVPR.2005.177>
- Farneback, G. (2003). Two-frame motion estimation based on polynomial expansion. *Scandinavian Conference on Image Analysis (SCIA)*, 2749, 363-370. [https://doi.org/10.1007/3-540-45103-X\\_50](https://doi.org/10.1007/3-540-45103-X_50)
- Fruin, J. J. (1971). *Pedestrian planning and design*. Metropolitan Association of Urban Designers and Environmental Planners.
- Jocher, G., Chaurasia, A., & Qiu, J. (2023). *Ultralytics YOLOv8*. Ultralytics. <https://github.com/ultralytics/ultralytics>
- Kuhn, H. W. (1955). The Hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1-2), 83-97. <https://doi.org/10.1002/nav.3800020109>
- Li, Y., Zhang, X., & Chen, D. (2018). CSRNet: Dilated convolutional neural networks for understanding the highly congested scenes. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 1091-1100.
- Page, E. S. (1954). Continuous inspection schemes. *Biometrika*, 41(1/2), 100-115. <https://doi.org/10.2307/2333009>
- Sindagi, V. A., Yasarla, R., & Patel, V. M. (2020). JHU-CROWD++: Large-scale crowd counting dataset and a benchmark method. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(5), 2594-2609.
- Simonyan, K., & Zisserman, A. (2015). Very deep convolutional networks for large-scale image recognition. *International Conference on Learning Representations*

(ICLR). <https://arxiv.org/abs/1409.1556>

Song, Q., Wang, C., Jiang, Z., Wang, Y., Tai, Y., Wang, C., Li, J., Huang, F., & Wu, Y. (2021). Rethinking counting and localization in crowds: A purely point-based framework. Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 3365-3374. <https://doi.org/10.1109/ICCV48922.2021.00335>

Stauffer, C., & Grimson, W. E. L. (1999). Adaptive background mixture models for real-time tracking. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2, 246-252.

Wang, B., Liu, H., Samaras, D., & Nguyen, M. H. (2020). Distribution matching for crowd counting. Advances in Neural Information Processing Systems (NeurIPS), 33, 1595-1607.

Xie, E., Wang, W., Yu, Z., Anandkumar, A., Alvarez, J. M., & Luo, P. (2021). SegFormer: Simple and efficient design for semantic segmentation with transformers. Advances in Neural Information Processing Systems (NeurIPS), 34, 12077-12090.

Zhang, Y., Zhou, D., Chen, S., Gao, S., & Ma, Y. (2016). Single-image crowd counting via multi-column convolutional neural network. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 589-597.

# APPENDICES

## APPENDIX 1: PROJECT FILE STRUCTURE

The complete project codebase is organised as follows:

```
project-root/
├── backend.py                # FastAPI server + 12-stage pipeline
├── orchestration
│   ├── analyzers.py         # DistanceAnalyzer, FlowAnalyzer,
│   │   TrajectoryTracker,
│   │   |
│   │   # CUSUMDetector, BehaviorClassifier,
│   │   fruin_los()
│   ├── zones.py             # DynamicZoneEngine, AdaptiveThresholdV2,
│   └── pressure
│       ├── walkable_area_estimator.py # Three-tier walkable estimation +
│       └── morphological_cleanup
│           ├── segformer_walkable.py # SegFormer-B2 ADE20K wrapper (FP16 CUDA)
│           ├── constants.py         # Grid dims, CUSUM params, ADE20K/COCO
│           └── class_sets
│               ├── database.py      # SQLite WAL persistence layer
│               ├── requirements.txt # Python dependencies
│               ├── Dockerfile       # Container definition
│               ├── Dockercompose.YML # Multi-container orchestration
│               ├── Nginx.conf       # Reverse proxy configuration
│               ├── index.html       # Single-page frontend shell
│               └── js/
│                   ├── app.js       # Application controller, WebSocket client
│                   ├── overlay.js   # Canvas2D ten-layer overlay renderer
│                   ├── panels.js    # Sidebar panel DOM update functions
│                   └── speech.js    # Web Speech API announcement system
└── CrowdCounting-P2PNet/
    ├── models/                # VGG16-BN backbone + prediction heads
    ├── weights/SHTechA.pth     # Pre-trained weights (MAE 51.96)
    ├── engine.py              # Training / evaluation loops
    ├── train.py               # Training entry point
    └── run_test.py            # Inference demonstration
```

## APPENDIX 2: INSTALLATION AND SETUP

1. Install GPU drivers (CUDA 12.1, cuDNN 8.9) following NVIDIA documentation for your OS.
2. `pip install -r requirements.txt` (see requirements.txt header for GPU vs CPU PyTorch wheels)
3. `git clone https://github.com/TencentYoutuResearch/CrowdCounting-P2PNet` into the project root
4. Download SHTechA.pth and place it in CrowdCounting-P2PNet/weights/
5. `uvicorn backend:app --host 0.0.0.0 --port 8000`
6. Open index.html in a browser (or serve with `python -m http.server 3000`)